



Modelado matemático

BCD5105 — III Cuatrimestre 2026

Actividad grupal clase 2

Modelos con ecuaciones diferenciales

Ficha de la actividad

Fecha de asignación:	Jueves 17 de setiembre de 2026 (clase 2)
Modalidad:	Grupal, en salas reducidas, con los grupos estables del proyecto integrador
Valor:	Tarea moral: no se entrega ni se califica
Duración estimada:	40 minutos en clase; lo que quede pendiente se termina durante la semana
Producto:	Un notebook de Colab compartido por el grupo, guardado en el Drive o el GitHub de cada quien

¿Qué es esta actividad?

Es la práctica que cierra la sesión sobre modelos con ecuaciones diferenciales. Tiene dos propósitos: leer a mano un modelo sencillo con las herramientas conceptuales de la pizarra (clasificar la ecuación, encontrar su equilibrio y decidir si es estable), y poner en marcha la herramienta principal del curso para EDO (`scipy.integrate.solve_ivp`) en tres casos concretos, incluido el ajuste a datos reales costarricenses.

Punto importante

Esta actividad es tarea moral: no se entrega ni tiene nota. Aun así, conviene tomarla en serio, porque todo lo que practican aquí es la base del avance del proyecto integrador (semana 8, 15%) y de la prueba parcial grupal (se asigna en la semana 7). El código limpio que produzcan hoy lo van a reutilizar.

Conexión con el resto del curso

Lo que practican hoy	Dónde vuelve a aparecer	Semana
Lectura conceptual de una EDO	Prueba parcial grupal (temas 1 a 6)	7 a 8
Simulación con <code>solve_ivp</code>	Avance del proyecto integrador	8
SIR con distintos R_0	Calibración bayesiana con PyMC	4
Ajuste a datos costarricenses	Entrega final y presentación del proyecto	15

Estructura de la actividad

Distribución del tiempo (40 minutos en total)

La actividad se desarrolla en cinco partes consecutivas dentro de la sala virtual del grupo.

Parte 0. Organización (3 minutos)

- Confirmar quiénes están en la sala.
- Crear el notebook compartido del grupo en Colab.
- Asignar quién lidera cada miniexploración.

Parte 1. Miniexploración a mano (8 minutos)

- Clasificar una EDO, hallar su equilibrio y dibujar su diagrama de fase.
- Transcribir las conclusiones al notebook en una celda de texto (Markdown).

Parte 2. Miniexploración numérica del logístico (10 minutos)

- Implementación con `solve_ivp` y comparación con la solución analítica.

Parte 3. Miniexploración SIR (10 minutos)

- Simulación de tres escenarios con valores distintos de R_0 y análisis comparativo.

Parte 4. Miniexploración con datos reales (7 minutos)

- Ajuste del modelo logístico a la serie poblacional del INEC y estimación de K .

Parte 5. Reflexión grupal (2 minutos)

- Celda de texto final con una observación matemática, una técnica y una pregunta abierta.

Parte 1: Miniexploración a mano

Ley de enfriamiento de Newton

Tiempo sugerido: 8 minutos

Esta parte es a mano, sin computadora. Uno de los integrantes trabaja en su iPad o cuaderno; los demás verifican cada paso. Al final, escriben las conclusiones en el notebook compartido como una celda de texto en Markdown.

Problema: la temperatura $T(t)$ de un objeto que se enfría en un ambiente a temperatura constante T_{amb} obedece la ley de Newton:

$$\frac{dT}{dt} = -k(T - T_{\text{amb}}),$$

donde $k > 0$ es una constante de proporcionalidad y $T(0) = T_0$ es la temperatura inicial.

Tareas (sin resolver la ecuación):

1. Clasifique la EDO: orden, ¿es lineal?, ¿es autónoma?
2. Traduzca la ecuación a una frase: ¿de qué depende la rapidez con la que cambia la temperatura?
3. Encuentre el equilibrio T^* (el valor donde $dT/dt = 0$) e interprételo físicamente.
4. Dibuje el diagrama de fase sobre la recta T : ¿hacia dónde apuntan las flechas cuando $T > T_{\text{amb}}$ y cuando $T < T_{\text{amb}}$? ¿El equilibrio es estable o inestable?
5. Bosqueje a mano $T(t)$ para un café a 85°C y para un refresco a 5°C en un cuarto a 22°C . ¿Qué tienen en común las dos curvas?
6. Opcional, para quien quiera la fórmula: resuelva por separación de variables y verifique que $T(t) = T_{\text{amb}} + (T_0 - T_{\text{amb}})e^{-kt}$. Con $k = 0.1 \text{ min}^{-1}$, ¿cuánto tarda el café en llegar a 40°C ?

Verificación

El equilibrio es $T^* = T_{\text{amb}}$ y es estable: por encima la temperatura baja, por debajo sube, y en ambos casos se acerca al ambiente. Si su diagrama de fase tiene las flechas apuntando hacia afuera, revisen el signo de $-k$.

Parte 2: El modelo logístico en código

Tiempo sugerido: 10 minutos

Implementen el modelo logístico de la pizarra con `solve_ivp` y comparen el resultado numérico con la solución analítica. Es un ejercicio de validación: si las dos curvas coinciden, sabemos que ambos métodos funcionan.

Tarea:

1. Implementar el campo vectorial del modelo logístico $\frac{dN}{dt} = rN(1 - N/K)$.

2. Resolver numéricamente con `solve_ivp` usando el método RK45.
3. Calcular la solución analítica con la fórmula vista en clase.
4. Graficar ambas curvas superpuestas en una sola figura.
5. Verificar visualmente que coinciden (la diferencia debería ser imperceptible).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Parametros del modelo
r = 0.3          # tasa intrinseca de crecimiento
K = 1000.0       # capacidad de carga
N0 = 10.0        # poblacion inicial

# 1. Solucion numerica con solve_ivp
def logistico(t, N):
    return r * N * (1 - N/K)

t_eval = np.linspace(0, 50, 500)
sol = solve_ivp(logistico, t_span=(0, 50), y0=[N0],
                t_eval=t_eval, method='RK45', rtol=1e-8)

# 2. Solucion analitica (la formula de la pizarra)
def analitica(t, K, r, N0):
    return K / (1 + ((K - N0)/N0) * np.exp(-r*t))

# 3. Comparacion
fig, ax = plt.subplots(figsize=(10, 5), layout='constrained')
ax.plot(sol.t, sol.y[0], label='solve_ivp (numerica)', color='#FF871E', lw=2)
ax.plot(t_eval, analitica(t_eval, K, r, N0), label='formula analitica',
        color='#262626', lw=1.5, linestyle='--')
ax.axhline(K, color='gray', linestyle=':', label=f'K = {K}')
ax.set_xlabel('tiempo'); ax.set_ylabel('poblacion N(t)')
ax.set_title('Modelo logistico: numerica vs analitica')
ax.legend(); ax.grid(alpha=0.3)
plt.show()
```

Pregunta para discutir en el grupo

Cambien N_0 a un valor mayor que K (por ejemplo $N_0 = 1500$). ¿Qué pasa con la curva? ¿Coincide con lo que predice el diagrama de fase de Verhulst que dibujamos en clase? Anoten su explicación en una celda de texto.

Parte 3: El modelo SIR con tres escenarios

Tiempo sugerido: 10 minutos

Aquí van a explorar cómo cambia la dinámica de una epidemia según el valor de R_0 . Es el ejercicio que más conecta con los carriles epidemiológicos del proyecto integrador.

Tarea:

1. Implementar el sistema SIR como un campo vectorial de tres componentes.
2. Simular tres escenarios:
 - Escenario A: $R_0 = 1.5$ (transmisión baja, como influenza estacional).
 - Escenario B: $R_0 = 2.5$ (transmisión media, como SARS-CoV-2 original).
 - Escenario C: $R_0 = 5.0$ (transmisión alta, como sarampión sin vacunación).
3. Para cada escenario, identificar el día del pico de infectados y la fracción total de la población que se infectó al final.
4. Graficar las tres curvas de infectados $I(t)$ en una misma figura para comparar.

```
def sir(t, y, beta, gamma):
    S, I, R = y
    N = S + I + R
    dS = -beta * S * I / N
    dI = beta * S * I / N - gamma * I
    dR = gamma * I
    return [dS, dI, dR]

# Condiciones iniciales
N = 1000
I0 = 1
S0 = N - I0
gamma = 0.1          # 10 dias para recuperarse

# Tres escenarios variando beta
escenarios = {
    'A: R0 = 1.5': 0.15,
    'B: R0 = 2.5': 0.25,
    'C: R0 = 5.0': 0.50,
}

fig, ax = plt.subplots(figsize=(11, 5), layout='constrained')

for etiqueta, beta in escenarios.items():
    sol = solve_ivp(sir, (0, 200), [S0, I0, 0], args=(beta, gamma),
                    t_eval=np.linspace(0, 200, 1000), rtol=1e-8)
    ax.plot(sol.t, sol.y[1], label=etiqueta, lw=2)
    pico = sol.t[np.argmax(sol.y[1])]
    total = N - sol.y[0][-1]
    print(f"{etiqueta} -> pico en dia {pico:.0f}, "
          f"infectados totales: {total:.0f} ({100*total/N:.1f} %)")

ax.set_xlabel('dia'); ax.set_ylabel('infectados activos')
```

```
ax.set_title('SIR: comparacion de tres escenarios de R0')
ax.legend(); ax.grid(alpha=0.3)
plt.show()
```

Pregunta clave

Observen el escenario C ($R_0 = 5.0$): la epidemia infecta a casi toda la población. ¿Es realista? ¿Qué le falta al modelo para capturar una epidemia tan transmisible como el sarampión en una población vacunada? Relacionen su respuesta con la fracción $1 - 1/R_0$ de la pizarra y anótenla en el notebook.

Parte 4: Ajuste del logístico a la población de Costa Rica

Tiempo sugerido: 7 minutos

La última miniexploración los conecta con datos reales. Van a ajustar el modelo logístico a la serie poblacional histórica del INEC y a estimar la capacidad de carga K implícita en los datos.

Tarea:

1. Usar los datos del INEC que ya están en el código.
2. Ajustar el modelo logístico con `scipy.optimize.curve_fit`.
3. Reportar el valor estimado de K (capacidad de carga) y de r (tasa de crecimiento). Comparen ese r con el $r \approx 0.022$ que estimamos a mano con dos censos: ¿por qué difieren?
4. Comparar K con el valor que el CCP-INEC proyecta para la población de Costa Rica en 2050.

```
from scipy.optimize import curve_fit

# Datos historicos del INEC: anios censales + estimacion 2025
anios = np.array([1864, 1892, 1927, 1950, 1973,
                  1984, 2000, 2011, 2022, 2025])
poblacion = np.array([120000, 243000, 472000, 800000, 1872000,
                      2417000, 3810000, 4301000, 5044197, 5191716])

# Trabajamos con t = anios - 1864 para que el tiempo empiece en 0
t = anios - anios[0]

# Modelo logistico (solucion analitica)
def logistico_analitico(t, K, r, N0):
    return K / (1 + ((K - N0)/N0) * np.exp(-r*t))

# Ajuste de parametros
params, _ = curve_fit(logistico_analitico, t, poblacion,
                      p0=[6e6, 0.03, 120000])
K_hat, r_hat, N0_hat = params

print(f"Capacidad de carga estimada K = {K_hat/1e6:.2f} millones")
print(f"Tasa de crecimiento r = {r_hat:.4f} por anio")
print(f"Poblacion inicial implicita = {N0_hat:.0f}")
```

```
# Visualizacion del ajuste
t_smooth = np.linspace(0, 200, 500)
fig, ax = plt.subplots(figsize=(11, 5), layout='constrained')
ax.scatter(anios, poblacion/1e6, color='#FF871E', s=60, zorder=3, label='
    Datos INEC')
ax.plot(anios[0] + t_smooth, logistico_analitico(t_smooth, *params)/1e6,
        color='#262626', lw=2, label='Modelo logistico ajustado')
ax.axhline(K_hat/1e6, color='gray', linestyle=':', label=f'K = {K_hat/1e6
    :.2f} M')
ax.set_xlabel('Anio'); ax.set_ylabel('Poblacion (millones)')
ax.set_title('Costa Rica: poblacion historica vs modelo logistico')
ax.legend(); ax.grid(alpha=0.3)
plt.show()
```

Comparación con la proyección oficial

El CCP-INEC estima que la población de Costa Rica en 2050 será de aproximadamente 5.7 millones, y que después iniciará un descenso. ¿Coincide su valor de K con esa proyección? Si difiere, discutan posibles razones: ¿es el modelo logístico demasiado simple para capturar el descenso poblacional que trae la transición demográfica?

Parte 5: Reflexión grupal

Tiempo sugerido: 2 minutos

Antes de cerrar el notebook, agreguen una celda de texto final (Markdown) con tres observaciones del grupo:

- **Una observación matemática:** ¿qué patrón notaron en la dinámica del SIR al variar R_0 ? ¿Cómo se compara la duración del brote entre los tres escenarios?
- **Una observación técnica:** ¿les pareció intuitivo `solve_ivp`? ¿Qué dificultad les tomó más tiempo (sintaxis, interpretación de `sol.y`, otra)?
- **Una pregunta abierta del grupo:** algo que les haya generado curiosidad y que les gustaría explorar más adelante para el proyecto integrador.

Qué hacer con el notebook

No se entrega, pero se guarda

1. Desde Colab: Archivo → Guardar una copia en Drive, o Guardar una copia en GitHub en el repositorio personal de cada integrante.
2. Usen un nombre reconocible, por ejemplo `Clase2_ED0_NombreDelGrupo.ipynb`, y listen a los integrantes en la primera celda.
3. Lo que no terminen en clase, complétenlo durante la semana: las partes 3 y 4 son el punto

de partida de varios carriles del proyecto.

4. En la puesta en común al final de la clase, cada grupo comparte en un minuto una de sus tres observaciones de la parte 5.

Consejos para que la actividad fluya

Sobre la distribución del trabajo

Trabajen en paralelo. Mientras uno resuelve la miniexploración 1 a mano, los otros pueden ir adelantando el código de la miniexploración 2 en el notebook compartido. Cuarenta minutos parecen muchos hasta que empiezan a depurar.

Sobre los errores de SciPy

Si `solve_ivp` arroja una advertencia de *step size too small*, revisen que los parámetros del modelo sean razonables. Un `beta` demasiado grande o un `N` demasiado pequeño hace que el sistema sea numéricamente difícil. Empiecen con los valores sugeridos antes de variarlos.

Sobre la conexión con el proyecto integrador

Si su grupo está pensando en un carril epidemiológico (dengue, COVID-19), la miniexploración 3 es directamente la base que van a extender. Si su proyecto es de dinámica poblacional o de adopción de productos (difusión de Bass, Sinpe Móvil, turismo), la miniexploración 4 les da el patrón de ajuste. Guarden el código limpio: lo van a reutilizar.

Sobre el uso de inteligencia artificial

Pueden consultar ChatGPT, Claude u otras herramientas durante la actividad, con dos condiciones: deben entender cada línea que copien (el profesor puede preguntar sobre cualquier parte del notebook en la próxima clase) y deben escribir ustedes mismos los comentarios en español que expliquen el código. Rige el Protocolo sobre el uso de la inteligencia artificial de Lead University. La IA acelera la sintaxis, pero el aprendizaje sigue siendo de ustedes.

Importante

- Esta actividad es tarea moral: no se entrega ni se califica.
- Una vez finalizadas las salas, el profesor abrirá una plenaria breve para la puesta en común.
- Las dudas técnicas se resuelven primero dentro del grupo, luego con el profesor por correo o en la atención al estudiante.